

APPENDIX A: CODE FOR MAXIMUM

Here we give the Mathematica code used for the rigorous verification of the Palacios conjecture for the maximum. It explicitly checks whether any graph attaining the maximal inverse degree index also attains the maximal symmetric division deg index, by intersecting the sets of extremal graphs.

```

ClearAll["Global`*"];

(*-----Vertex degrees from edge list-----*)
VertexDegrees[edges_List,n_Integer:0]:=Module[{maxv,deg},
maxv=If[n>0,n,Max[Flatten[edges]]];
deg=ConstantArray[0,maxv];
Do[deg[[e[[1]]]]+=1;deg[[e[[2]]]]+=1,{e,edges}];
deg];

(*-----Inverse degree index I(G)-----*)
Index[edges_List]:=Module[{deg,n},n=Max[Flatten[edges]];
deg=VertexDegrees[edges,n];
Total[1/deg]];

(*-----SDD(G)-----*)
SDD[edges_List]:=Module[{deg,n},n=Max[Flatten[edges]];
deg=VertexDegrees[edges,n];
Total[Table[With[{a=deg[[e[[1]]]],b=deg[[e[[2]]]]},
Min[a,b]/Max[a,b]+Max[a,b]/Min[a,b]],{e,edges}]]];

(*-----All c-cyclic graphs with Delta=n-1-----*)
AllGraphs[n_Integer,c_Integer]:=Module[{base,others,allExtraEdges,extraSubsets},
others=Range[2,n];
base=Table[{1,i},{i,others}];
allExtraEdges=Subsets[others,{2}];
extraSubsets=Subsets[allExtraEdges,{c}];
Map[Join[base,#]&,extraSubsets]];

(*-----Explicit construction of Bianchi maximal graphs-----*)
BianchiGraph[c_Integer,n_Integer,k_Integer]:=Module[{u=1,edges,v,w,x,y},
Which[
c==1,(*H_{n,1}*)v=2;w={3};
edges=Join[Table[{u,i},{i,2,n}],{{v,w[[1]]}}],
c==2,(*H_{n,2}*)v=2;w={3,4};
edges=Join[Table[{u,i},{i,2,n}],{{v,w[[1]]},{v,w[[2]]}}],
c==3,If[k==1,(*H_{n,3}:4,2,2,2*)v=2;w=Range[3,5];
edges=Join[Table[{u,i},{i,2,n}],Table[{v,w[[i]]},{i,1,3}]],
(*k=2:triangle (3,3,3)*)v={2,3,4};
edges=Join[Table[{u,i},{i,2,n}],{{v[[1]],v[[2]]},{v[[2]],v[[3]]},{v[[3]],v[[1]]}}],
c==4,If[k==1,(*H_{n,4}:5,2,2,2,2*)v=2;w=Range[3,6];
edges=Join[Table[{u,i},{i,2,n}],Table[{v,w[[i]]},{i,1,4}]],
(*k=2:4,3,3,2*)v=2;w={3,4};x=5;
edges=Join[Table[{u,i},{i,2,n}],{{v,w[[1]]},{v,w[[2]]},{w[[1]],w[[2]]},{v,x}}],
c==5,If[k==1,(*6,2^5*)v=2;w=Range[3,7];
edges=Join[Table[{u,i},{i,2,n}],Table[{v,w[[i]]},{i,1,5}]],
If[k==2,(*5,3,3,2,2*)v=2;w={3,4};x={5,6};
edges=Join[Table[{u,i},{i,2,n}],{{v,w[[1]]},{v,w[[2]]},{w[[1]],w[[2]]},{v,x[[1]]},{v,x[[2]]}}],
(*k=3:4,4,3,3*)v={2,3};w={4,5};
edges=Join[Table[{u,i},{i,2,n}],{{v[[1]],v[[2]]},{v[[1]],w[[1]]},{v[[1]],w[[2]]},{v[[2]],
w[[1]]},{v[[2]],w[[2]]}}],
c==6,If[k==1,(*7,2^6*)v=2;w=Range[3,8];
edges=Join[Table[{u,i},{i,2,n}],Table[{v,w[[i]]},{i,1,6}]],
If[k==2,(*6,3,3,2,2,2*)v=2;w={3,4};x={5,6,7};

```

```

edges=Join[Table[{u,i},{i,2,n}],{{v,w[[1]]},{v,w[[2]]},{w[[1]],w[[2]]},{v,x[[1]]},{v,x[[2]]},
  {v,x[[3]]}}],
If[k==3,(*5,4,3,3,2 { corrected*})v=2; w={3,4,5,6};
edges=Join[Table[{u,i},{i,2,n}],{{2,3},{2,4},{2,5},{2,6},{3,4},{3,5}}],
(*k=4:4,4,4,4*)v={2,3,4,5};
edges=Join[Table[{u,i},{i,2,n}],{{v[[1]],v[[2]]},{v[[1]],v[[3]]},{v[[1]],v[[4]]},{v[[2]],
  v[[3]]},{v[[2]],v[[4]]},{v[[3]],v[[4]]}}]]];
edges];

(*-----Verification of the conjecture-----*)
CheckConjecture[c_Integer,nMax_Integer]:=Module[{n,graphs,iVals,sVals,
  maxI,maxS,maxIgraphs,maxSgraphs,vecs,vecGraphs,vecI,vecS,sameMaxGraphQ},
Print["===== c = ",c," ====="];
Do[If[n<c+2,Continue[]];
graphs=AllGraphs[n,c];
If[Length[graphs]==0,Continue[]];
iVals=Index/@graphs;
sVals=SDD/@graphs;
maxI=Max[iVals];
maxS=Max[sVals];
maxIgraphs=Pick[graphs,iVals,maxI];
maxSgraphs=Pick[graphs,sVals,maxS];
Print["n=",n," (",Length[graphs]," graphs)"];
Print[" max I = ",maxI," = ",Rationalize[maxI]];
Print[" max SDD = ",maxS," = ",Rationalize[maxS]];
(*Definition of Bianchi vectors (degree sequences)*)
vecs=Which[
c==1,{{n-1,2,2}~Join~Table[1,{n-3}]},
c==2,{{n-1,3,2,2}~Join~Table[1,{n-4}]},
c==3,{{n-1,4,2,2,2}~Join~Table[1,{n-5}],{n-1,3,3,3}~Join~Table[1,{n-4}]},
c==4,{{n-1,5,2,2,2,2}~Join~Table[1,{n-6}],{n-1,4,3,3,2}~Join~Table[1,{n-5}]},
c==5,{{n-1,6,2,2,2,2,2}~Join~Table[1,{n-7}],{n-1,5,3,3,2,2}~Join~Table[1,{n-6}],
  {n-1,4,4,3,3}~Join~Table[1,{n-5}]},
c==6,{{n-1,7,2,2,2,2,2,2}~Join~Table[1,{n-8}],{n-1,6,3,3,2,2,2}~Join~Table[1,{n-7}],
  {n-1,5,4,3,3,2}~Join~Table[1,{n-6}],{n-1,4,4,4,4}~Join~Table[1,{n-5}]}
];
vecGraphs=Table[BianchiGraph[c,n,k],{k,1,Length[vecs]}];
vecI=Index/@vecGraphs;
vecS=SDD/@vecGraphs;
Print[" Values for Bianchi maximal vectors:"];
Do[
Print[" Vector ",i," : ",vecs[[i]]," I=",vecI[[i]]," SDD=",vecS[[i]]];
If[vecI[[i]]==maxI,Print[" <-- attains max I"]];
If[vecS[[i]]==maxS,Print[" <-- attains max SDD"]];,
{i,Length[vecs]}
];
sameMaxGraphQ=Length[Intersection[maxIgraphs,maxSgraphs]]>0;
Print[" Max I and Max SDD on the same graph? ",sameMaxGraphQ];
If[!sameMaxGraphQ,
Print[" *** WARNING: Palacios conjecture DOES NOT HOLD for this case!"],
Print[" (Conjecture currently holds.)"]
];
Print[""];
{n,c+2,nMax}
];

(*-----Execution-----*)
Do[CheckConjecture[c,c+5],{c,1,6}]

```

APPENDIX B: CODE FOR MINIMUM

```

ClearAll["Global`*"];

(*-----Vertex degrees from edge list-----*)
VertexDegrees[edges_List,n_Integer:0]:=Module[{maxv,deg},
maxv=If[n>0,n,Max[Flatten[edges]]];
deg=ConstantArray[0,maxv];
Do[deg[[e[[1]]]]+=1;deg[[e[[2]]]]+=1,{e,edges}];
deg];

(*-----Inverse degree index I(G)-----*)
Index[edges_List]:=Module[{deg,n},n=Max[Flatten[edges]];
deg=VertexDegrees[edges,n];
Total[1/deg]];

(*-----SDD(G)-----*)
SDD[edges_List]:=Module[{deg,n},n=Max[Flatten[edges]];
deg=VertexDegrees[edges,n];
Total[Table[With[{a=deg[[e[[1]]]],b=deg[[e[[2]]]]},
Min[a,b]/Max[a,b]+Max[a,b]/Min[a,b]],{e,edges}]]];

(*-----Check connectivity of graph given by edge list-----*)
ConnectedQ[edges_List,n_Integer]:=Module[{adj,visited,queue,v},
adj=Table[{},{n}];
Do[AppendTo[adj[[e[[1]]]],e[[2]];AppendTo[adj[[e[[2]]]],e[[1]],{e,edges}];
visited=ConstantArray[False,n];
queue={1};
visited[[1]]=True;
While[queue!={},v=First[queue];
queue=Rest[queue];
Do[If[!visited[[w]],visited[[w]]=True;AppendTo[queue,w],{w,adj[[v]]}]]];
AllTrue[visited,TrueQ]];

(*-----All connected graphs with n vertices and m edges-----*)
AllConnectedGraphs[n_Integer,m_Integer]:=Module[{allEdges,edgeSets},
allEdges=Subsets[Range[n],{2}];
edgeSets=Subsets[allEdges,{m}];
Select[edgeSets,ConnectedQ[#,n]&]];

(*-----All c-cyclic graphs of order n-----*)
AllCCyclicGraphs[n_Integer,c_Integer]:=AllConnectedGraphs[n,n+c-1];

(*-----Degree sequence sorted descending-----*)
DegreeSequence[edges_List,n_Integer]:=Sort[VertexDegrees[edges,n],Greater];

(*-----Maximum number of subsets we allow-----*)
maxSubsets=500000;

(*-----Main routine for checking minimum-----*)
CheckMinimumConjecture[c_Integer,nMax_Integer]:=
Module[{n,m,totalSubsets,graphs,iVals,sVals,minI,minS,
minIpos,minSpos,seqI,seqS,sI},
Print["===== c = ",c," ====="];
Do[m=n+c-1;
totalSubsets=Binomial[Binomial[n,2],m];
If[totalSubsets>maxSubsets,
Print["n = ",n," : skipping (",totalSubsets," subsets > ",maxSubsets,")"];
Continue[]];
Print["n = ",n," (",totalSubsets," subsets) ..."];
graphs=AllCCyclicGraphs[n,c];

```

```

Print[" number of connected graphs = ",Length[graphs]];
If[Length[graphs]==0,Continue[]];
iVals=Index/@graphs;
sVals=SDD/@graphs;
minI=Min[iVals];
minS=Min[sVals];
minIpos=First[FirstPosition[iVals,minI]];
minSpos=First[FirstPosition[sVals,minS]];
seqI=DegreeSequence[graphs[[minIpos]],n];
seqS=DegreeSequence[graphs[[minSpos]],n];
sI=sVals[[minIpos]];
Print[" min I = ",minI," SDD(min I) = ",sI," sequence: ",seqI];
Print[" min SDD = ",minS," sequence: ",seqS];
If[sI==minS,
Print[" \[Checkmark] Same SDD values \[Dash] minima coincide."],
Print[" \[Cross] DIFFERENT SDD values \[Dash] COUNTEREXAMPLE!"]];
If[seqI==seqS,
Print[" \[Checkmark] Same degree sequence \[Dash] conjecture holds."],
Print[" \[Cross] DIFFERENT degree sequences \[Dash] conjecture DOES NOT HOLD."]];
Print[""];{n,c+2,nMax}}];

(*-----EXECUTION for c=3,4,5,6-----*)
Do[CheckMinimumConjecture[c,7],{c,3,6}]

```

APPENDIX C: CODE FOR MINIMUM FOR $c = 6, n = 8$

```

ClearAll["Global`*"];

(*-----Vertex degrees from edge list-----*)
VertexDegrees[edges_List,n_Integer:0]:=Module[{maxv,deg},
maxv=If[n>0,n,Max[Flatten[edges]]];
deg=ConstantArray[0,maxv];
Do[deg[[e[[1]]]]+=1;deg[[e[[2]]]]+=1,{e,edges}];
deg];

(*-----Inverse degree index I(G)-----*)
Index[edges_List]:=Module[{deg,n},n=Max[Flatten[edges]];
deg=VertexDegrees[edges,n];
Total[1/deg]];

(*-----SDD(G)-----*)
SDD[edges_List]:=Module[{deg,n},n=Max[Flatten[edges]];
deg=VertexDegrees[edges,n];
Total[Table[With[{a=deg[[e[[1]]]],b=deg[[e[[2]]]]},
Min[a,b]/Max[a,b]+Max[a,b]/Min[a,b]],{e,edges}]]];

(*-----Check connectivity (BFS)-----*)
ConnectedQ[edges_List,n_Integer]:=Module[{adj,visited,queue,v},
adj=Table[{},{n}];
Do[AppendTo[adj[[e[[1]]]],e[[2]]];
AppendTo[adj[[e[[2]]]],e[[1]]],{e,edges}];
visited=ConstantArray[False,n];
queue={1};
visited[[1]]=True;
While[queue!={} ,v=First[queue];queue=Rest[queue];
Do[If[!visited[[w]],visited[[w]]=True;AppendTo[queue,w]],{w,adj[[v]]}]];
AllTrue[visited,TrueQ]];

(*-----Backtracking: all realizations of a degree sequence-----*)
AllGraphsFromDegreeSequence[degseq_List]:=

```

```

Module[{n=Length[degseq],results={}},
ClearAll[backtrack];
backtrack[deg_,edges_]:=Module[{i,d,others,subset,newDeps,newEdges},
If[AllTrue[deg,##==0&],
AppendTo[results,Sort[edges]];
Return[]
];
i=First[Ordering[deg,-1]];
d=deg[[i]];
others=Select[Range[n],#!=i&&deg[[#]]>0&];
If[Length[others]<d,Return[]];
Do[subset=Sort[s];
newDeps=deg;
newDeps[[i]]=0;
newEdges=edges;
Do[newDeps[[v]]=newDeps[[v]]-1;
AppendTo[newEdges,{i,v}},{v,subset}];
If[Min[newDeps]>=0,backtrack[newDeps,newEdges]],
{s,Subsets[others,{d}]]}
];
backtrack[degseq,{}];
results
];

(*-----MAIN PART-----*)
n=8;
c=6;
m=n+c-1;(*13*)

(*All possible degree sequences for n=8,m=13*)
allSeq=IntegerPartitions[2*m,{n},Range[1,n-1]];
(*already nonincreasing, elements 1..7*)
Print["Number of candidate degree sequences: ",Length[allSeq]];

results={};
Do[
seq=seqcand;
graphs=AllGraphsFromDegreeSequence[seq];
conn=Select[graphs,ConnectedQ[#,n]&];
If[Length[conn]>0,
sddVals=SDD/@conn;
minSDD=Min[sddVals];
AppendTo[results,{seq,minSDD,Length[conn]}];
Print[seq," -> min SDD = ",minSDD," (",Length[conn]," graphs)"];
],
{seqcand,allSeq}
];

(*Smallest SDD*)
If[results!={},
sorted=SortBy[results,#[[2]]&];
Print["\n=== RESULTS ==="];
Print["Global minimum SDD: ",sorted[[1,2]]," for sequence ",sorted[[1,1]]];
Print["Smallest SDD among minimal I-sequence (4,4,3,3,3,3,3,3) is 53/2 = ",53/2];
Print["All sequences and their minSDD:"];
TableForm[sorted,TableHeadings->{None,{"sequence","min SDD","number of graphs"}}]
,
Print["No sequence has a connected realization! (Impossible)"]
]

```

APPENDIX D: CODE FOR MINIMUM FOR $c = 6, n = 9$

```

ClearAll["Global`*"];

(*-----Vertex degrees from edge list-----*)
VertexDegrees[edges_List,n_Integer:0]:=Module[{maxv,deg},
maxv=If[n>0,n,Max[Flatten[edges]]];
deg=ConstantArray[0,maxv];
Do[deg[[e[[1]]]]+=1;deg[[e[[2]]]]+=1,{e,edges}];
deg];

(*-----Inverse degree index I(G)-----*)
Index[edges_List]:=Module[{deg,n},n=Max[Flatten[edges]];
deg=VertexDegrees[edges,n];
Total[1/deg]];

(*-----SDD(G)-----*)
SDD[edges_List]:=Module[{deg,n},n=Max[Flatten[edges]];
deg=VertexDegrees[edges,n];
Total[Table[With[{a=deg[[e[[1]]]],b=deg[[e[[2]]]]],
Min[a,b]/Max[a,b]+Max[a,b]/Min[a,b]],{e,edges}]]];

(*-----Check connectivity (BFS)-----*)
ConnectedQ[edges_List,n_Integer]:=Module[{adj,visited,queue,v},
adj=Table[{},{n}];
Do[AppendTo[adj[[e[[1]]]],e[[2]]];
AppendTo[adj[[e[[2]]]],e[[1]]],{e,edges}];
visited=ConstantArray[False,n];
queue={1};
visited[[1]]=True;
While[queue!={},v=First[queue];queue=Rest[queue];
Do[If[!visited[[w]],visited[[w]]=True;AppendTo[queue,w]],{w,adj[[v]]}]];
AllTrue[visited,TrueQ]];

(*-----Backtracking: all realizations of a degree sequence-----*)
AllGraphsFromDegreeSequence[degseq_List]:=
Module[{n=Length[degseq],results={}},
ClearAll[backtrack];
backtrack[deg_,edges_]:=Module[{i,d,others,subset,newDeps,newEdges},
If[AllTrue[deg,#==0&],
AppendTo[results,Sort[edges]];
Return[]];
i=First[Ordering[deg,-1]];
d=deg[[i]];
others=Select[Range[n],#!=i&&deg[[#]]>0&];
If[Length[others]<d,Return[]];
Do[subset=Sort[s];
newDeps=deg;
newDeps[[i]]=0;
newEdges=edges;
Do[newDeps[[v]]=newDeps[[v]]-1;
AppendTo[newEdges,{i,v}},{v,subset}];
If[Min[newDeps]>=0,backtrack[newDeps,newEdges]],
{s,Subsets[others,{d]}]
];
backtrack[degseq,{)];
results
];

(*-----MAIN PART-----*)

```

```

n=9;
c=6;
m=n+c-1;(*14*)
(*All possible degree sequences for n=9,m=14*)
allSeq=IntegerPartitions[2*m,{n},Range[1,n-1]];
(*already nonincreasing, elements 1..8*)
Print["Number of candidate degree sequences: ",Length[allSeq]];

results={};
Do[
seq=seqcand;
graphs=AllGraphsFromDegreeSequence[seq];
conn=Select[graphs,ConnectedQ[#,n]&];
If[Length[conn]>0,
sddVals=SDD/@conn;
minSDD=Min[sddVals];
AppendTo[results,{seq,minSDD,Length[conn]}];
Print[seq," -> min SDD = ",minSDD," (",Length[conn]," graphs)"];
],
{seqcand,allSeq}
];

(*Smallest SDD*)
If[results!={},
sorted=SortBy[results,#[[2]]&];
Print["\n=== RESULTS ==="];
Print["Global minimum SDD: ",sorted[[1,2]]," for sequence ",sorted[[1,1]]];
Print["Smallest SDD among minimal I-sequence (4,3,3,3,3,3,3,3) is ",
SDD[AllGraphsFromDegreeSequence[{4,3,3,3,3,3,3,3}][[1]]];
Print["All sequences and their minSDD:"];
TableForm[sorted,TableHeadings->{None,{"sequence","min SDD","number of graphs"}}]
,
Print["No sequence has a connected realization! (Impossible)"]
]

```

APPENDIX E: CASE $c = 7$

In this appendix we give the complete source code used for the computer analysis of heptacyclic graphs ($c = 7$).

```

(* ----- Basic functions ----- *)
ClearAll["Global'*"];

VertexDegrees[edges_List, n_Integer : 0] := Module[{maxv, deg},
maxv = If[n > 0, n, Max[Flatten[edges]]];
deg = ConstantArray[0, maxv];
Do[deg[[e[[1]]]] += 1; deg[[e[[2]]]] += 1, {e, edges}];
deg];

Iindex[edges_List] := Module[{deg, n},
n = Max[Flatten[edges]];
deg = VertexDegrees[edges, n];
Total[1/deg]];

SDD[edges_List] := Module[{deg, n},
n = Max[Flatten[edges]];
deg = VertexDegrees[edges, n];
Total[Table[With[{a = deg[[e[[1]]]], b = deg[[e[[2]]]]},
Min[a, b]/Max[a, b] + Max[a, b]/Min[a, b]], {e, edges}]]];

```

```

(* ----- Construction of all c-cyclic graphs with a universal vertex ----- *)
AllGraphs[n_Integer, c_Integer] := Module[{base, others, allExtraEdges, extraSubsets},
  others = Range[2, n];
  base = Table[{1, i}, {i, others}];
  allExtraEdges = Subsets[others, {2}];
  extraSubsets = Subsets[allExtraEdges, {c}];
  Map[Join[base, #] &, extraSubsets]];

(* ----- Havel-Hakimi graphicality check ----- *)
HavelHakimiQ[seq_List] := Module[{d = Select[seq, # > 0 &]},
  While[True,
    If[Length[d] == 0, Return[True]];
    d = Sort[d, Greater];
    If[d[[1]] >= Length[d], Return[False]];
    Do[d[[i + 1]] = d[[i + 1]] - 1, {i, d[[1]]}];
    d = Rest[d];
    If[Min[d] < 0, Return[False]];
  ];

(* ----- All degree sequences of subgraph H (without isolated vertices) ----- *)
HDegreeSequences[c_Integer] := Module[{parts},
  parts = IntegerPartitions[2 c, All, Range[c]]; (* parts between 1 and c *)
  Sort[#, Greater] & /@ parts
];

(* ----- Asymptotic functional A(H) ----- *)
Afunctional[degrees_List] := Module[{d = degrees, k},
  k = Length[d];
  Total[1/(1 + d)] - k
];

(* ----- Candidate degree sequences for G ----- *)
CandidateSequences[degrees_List, n_Integer] := Module[{k = Length[degrees], leaves},
  leaves = Max[0, n - 1 - k]; (* leaves of degree 1 *)
  Join[{n - 1}, (1 + #) & /@ degrees, ConstantArray[1, leaves]]
];

(* ----- Patterns from Remark 6, Bianchi et al. (2015) ----- *)
PatternV1[c_, n_] := Join[{n - 1, c + 1}, ConstantArray[2, c],
  ConstantArray[1, n - c - 2]];
PatternV2[c_, n_] := Join[{n - 1, c}, {3, 3}, ConstantArray[2, c - 3],
  ConstantArray[1, n - c - 1]];
PatternV3[c_, n_] := Join[{n - 1, c - 1}, {4, 3, 3},
  ConstantArray[2, c - 5], ConstantArray[1, n - c]];

(* ----- Main analysis for a given c ----- *)
c = 7; (* <-- change as needed *)
Print["====="];
Print["Analysis for c = ", c];
Print["====="];

(* 1. All degree sequences of subgraph H *)
hSeqs = HDegreeSequences[c];
Print["Number of partitions of 2c with parts <= c: ", Length[hSeqs]];

(* 2. Compute A(H) for graphical ones *)
candidates = {};
Do[
  seq = hSeqs[[i]];
  If[HavelHakimiQ[seq],
    AppendTo[candidates, {seq, Afunctional[seq]}];

```



```

],
{i, Length[hSeqs]}
];
Print["Of which are graphical: ", Length[candidates]];

(* 3. Maximum value of A(H) *)
maxA = Max[candidates[[All, 2]]];
bestSeqs = Select[candidates, #[[2]] == maxA &];
Print["Maximum A(H) = ", maxA];
Print["Optimal degree sequences for H (nonincreasing):"];
bestSeqs // TableForm

(* 4. Example sequence for G *)
nSmall = c + 2;
Print["\nCorresponding degree sequences for G (n = ", nSmall, "):"];
Do[
hseq = bestSeqs[[i, 1]];
gseq = CandidateSequences[hseq, nSmall];
Print["H = ", hseq, " -> G = ", gseq],
{i, Length[bestSeqs]}
];

(* 5. Comparison with patterns from Remark 6 *)
Print["\nValues of A for patterns from Remark 6 (c = ", c, "):"];

If[c >= 5,
Module[{hDegreesV3},
hDegreesV3 = Join[{c - 2}, {3}, {2, 2}, ConstantArray[1, c - 5]];
Print["v3 (n-1, c-1, 4,3,3, 2^{c-5}, 1^{...}): ",
PatternV3[c, nSmall], " A = ", Afunctional[hDegreesV3]]
]
];

Module[{hDegreesV2},
hDegreesV2 = Join[{c - 1}, {2, 2}, ConstantArray[1, c - 3]];
Print["v2 (n-1, c, 3,3, 2^{c-3}, 1^{...}): ",
PatternV2[c, nSmall], " A = ", Afunctional[hDegreesV2]]
];

Module[{hDegreesV1},
hDegreesV1 = Join[{c}, ConstantArray[1, c]];
Print["v1 (n-1, c+1, 2^c, 1^{...}): ",
PatternV1[c, nSmall], " A = ", Afunctional[hDegreesV1]]
];

(* 6. Optional enumeration for small n *)
nMaxEnumerate = 10; (* for c=7, n=9 and n=10 are feasible *)
Print["\n====="];
Print["Enumeration of all graphs with a universal vertex for c = ", c,
" and n from ", c+2, " to ", nMaxEnumerate];
Print["====="];

Do[
If[n < c + 2, Continue[]];
totalSubsets = Binomial[Binomial[n - 1, 2], c];
If[totalSubsets > 20000000,
Print["n = ", n, " : too many subsets (", totalSubsets,
"), skipping."];
Continue[]
];
Print["\nGenerating graphs for n = ", n, " (", totalSubsets,

```

```

" subsets) ..."];
graphs = AllGraphs[n, c];
Print[" number of connected? (all are connected) = ", Length[graphs]];
iVals = Iindex /@ graphs;
sVals = SDD /@ graphs;
maxI = Max[iVals];
maxS = Max[sVals];
maxIgraphs = Pick[graphs, iVals, maxI];
maxSgraphs = Pick[graphs, sVals, maxS];
Print[" max I = ", maxI, " = ", Rationalize[maxI]];
Print[" max SDD = ", maxS, " = ", Rationalize[maxS]];
seqsMaxI = Union[Sort[VertexDegrees[#, n], Greater] & /@ maxIgraphs];
seqsMaxS = Union[Sort[VertexDegrees[#, n], Greater] & /@ maxSgraphs];
Print[" Degree sequence(s) with max I: ", seqsMaxI];
Print[" Degree sequence(s) with max SDD: ", seqsMaxS];
If[seqsMaxI == seqsMaxS,
Print[" *** Maxima coincide on the same degree sequence! ***"],
Print[" --- Maxima are DIFFERENT ---"]
];
, {n, c + 2, nMaxEnumerate}
]

```